

POCKETSMART AI: CODE LAYOUT, READABILITY AND REUSABILITY REPORT

1. Architectural Code Layout & Modular Directory Structure

The PocketSmart AI codebase strictly adheres to modular software engineering principles by separating routing, presentation, business logic, data models, and authentication into distinct directories:

PocketSmart_AI/

- |— app.py # ASGI entry point, CORS middleware, and root route controllers
- |— auth.py # OAuth2 JWT token handlers and user session tracking
- |— users.json # Persistent flat-file user credential storage
- |— models/
 - | — schemas.py # Pydantic data schemas and request body validation
- |— routes/
 - | — planners.py # Dedicated APIRouter controllers for Home, Party, and Jewelry
- |— services/
 - | — gemini_utils.py # LLM integration layer and fallback calculation engine
- |— static/
 - | — css/styles.css # Custom stylesheet overrides and responsive layout styling

```

|   └─ uploads/           # Multipart image upload buffer for
multimodal vision queries
└─ templates/             # Jinja2 server-side rendered HTML views
    └─ index.html, login.html, register.html, dashboard.html
        └─ home_planner.html, party_planner.html,
jewelry_planner.html, history.html

```

2. Code Readability & Standards Compliance

- **PEP 8 Compliance:** All backend Python files follow PEP 8 standards, utilizing 4-space indentation, explicit snake_case naming for functions/variables, and PascalCase for Pydantic schema classes.
- **Type Annotations & Schema Integrity:** Extensive type hinting (Optional[str], float, int, BaseModel) is implemented across all API route signatures, preventing runtime typing ambiguities and enabling automatic OpenAPI documentation generation.
- **Self-Documenting Code & Inline Explanations:** Critical routing blocks, mathematical percentage allocations, and exception handling blocks include clean inline comments and docstrings.

3. Reusability & Decoupled Design Patterns

- **FastAPI APIRouter Modularity:** The core planning engine is separated into routes/planners.py using APIRouter(), allowing modular scaling and independent route testing without modifying the core app.py server gateway.
- **Reusable Helper Functions:**
 - `save_upload_file()`: Generalized file-saving utility with unique timestamp prefixes for uploaded media.

- `create_access_token()`: Centralized JWT encoding utility accepting arbitrary token payload dictionaries and configurable expiration times.
- `load_users()` & `save_users()`: Atomic file I/O operations abstracting credential storage for any authentication endpoint.
- **Frontend Component Reusability:** Jinja2 templates utilize standardized Bootstrap 5 cards, input controls, and unified navigation headers, ensuring a consistent user experience across all three planner interfaces.